Manual de instalacion y entorno de desarrollo

Version 1.0 | feature/terreno-agenda-visitas | d0735003f88a6b6a5d540049866bd74a680c2e04

2026-10-19 14:31 Hora verano Sudamérica Pacífico

DOCUMENTACION TECNICA Y PEDAGOGICA

Proyecto: Puertas Blindadas Documento: 02 Version: 1.0 Fecha de corte: 2026-10-19 14:31 Hora verano Sudamérica Pacífico
Rama: `feature/terreno-agenda-visitas` SHA documentado: `d0735003f88a6b6a5d540049866bd74a680c2e04`

> Alcance de evidencia: este manual describe el codigo, schema Prisma, migraciones, pruebas y documentacion presentes en el SHA indicado. Cuando una capacidad no puede demostrarse desde esas fuentes se marca como NO CONFIRMADO EN EL REPOSITORIO. Implementado, preparado, bloqueado, legacy y pendiente son estados distintos.

Tabla de contenidos

Tabla de contenidos 2
Como leer este documento 3
Requisitos verificados 3
Primera instalacion desde cero 3
Comandos explicados 3
Actualizar una copia existente 3
Variables de entorno 3
Verificacion basica 4
Errores frecuentes 4
Fuentes de verdad consultadas 4

Como leer este documento

- Comportamiento actual significa que existe codigo operativo y evidencia verificable.
- Preparado significa que existen contratos, modelos o interfaz, pero falta una dependencia o activacion.
- Bloqueado significa que el sistema impide deliberadamente la operacion por seguridad o falta de autorizacion.
- Pendiente significa que no existe una implementacion completa demostrable.
- Las rutas de archivos se expresan desde `CODIGO/`.

Requisitos verificados

El repositorio contiene dos espacios npm: `Controladores/` (Node.js, Express, TypeScript, Prisma) y `Vistas/` (React, TypeScript, Vite). PostgreSQL es la fuente persistente. Docker aparece en la documentacion de despliegue, pero el desarrollo local puede usar PostgreSQL instalado o un contenedor autorizado.

Primera instalacion desde cero

1. Instalar Git, Node.js LTS, npm y PostgreSQL compatible.
2. Clonar el repositorio y seleccionar la rama documentada.
3. Crear archivos `.env` locales a partir de la configuracion esperada; nunca versionar secretos.
4. Ejecutar `npm install` en `Controladores/` y `Vistas/`.
5. Configurar `DATABASE_URL` para una base local no productiva.
6. En `Controladores/`, ejecutar `npx prisma generate` y luego `npx prisma migrate deploy` solo sobre la base local autorizada.
7. Levantar backend con `npm run dev` y frontend con `npm run dev`.

Comandos explicados

Comando	Que hace	Resultado esperado	Si falla
`git clone <url>`	Copia historia y archivos	Carpeta con `.git`	Verificar URL, acceso y red
`git switch feature/terreno-agenda-visitas`	Selecciona la rama	Nombre de rama confirmado	Ejecutar `git fetch origin` y comprobar existencia
`npm install`	Resuelve dependencias del package.json	`node_modules` local	Revisar version Node, proxy y lockfile
`npx prisma generate`	Genera cliente Prisma	Cliente actualizado	Validar schema y `DATABASE_URL`
`npx prisma migrate deploy`	Aplica migraciones pendientes	Lista aplicada sin drift	Detenerse ante drift; no usar reset
`npm run dev`	Inicia servicio con recarga	Puerto local escuchando	Revisar `.env`, puerto y logs
`npm run build`	Verifica TypeScript y empaqueta	Codigo de salida 0	Resolver primer error, no ocultarlo

Actualizar una copia existente

Confirmar que el arbol este limpio, descargar referencias, revisar cambios remotos y actualizar la rama sin borrar trabajo local. Si hay cambios propios, no usar `reset --hard`; resolver la situacion de forma consciente.

Variables de entorno

`DATABASE_URL` es obligatoria para Prisma. Las variables documentales y de almacenamiento deben inyectarse por proceso y nunca quedar en Git. Consultar los modulos de configuracion antes de inventar nombres.

Verificacion basica

- Backend compila.
- Frontend compila.
- Prisma valida.
- La ruta de salud responde si esta configurada.
- El login usa una cuenta sintetica o autorizada.
- Ningun proceso apunta a Clouding o RRFF.

Errores frecuentes

- `P2022`: la base no tiene una columna esperada; comprobar migraciones y no aplicar arreglos improvisados.
- Puerto ocupado: identificar el proceso, no matar servicios ajenos.
- 403: revisar CU efectivo, perfil, configuracion particular y alcance.
- Pantalla vacia: inspeccionar la respuesta HTTP antes de asumir ausencia de datos.

Fuentes de verdad consultadas

- `Controladores/prisma/schema.prisma`
- `Controladores/pruebas`
- `Controladores/src/controladores/C_Finanzas.ts`
- `Controladores/src/rutas/finanzas.ts`
- `Vistas/src`
- `Vistas/src/App.tsx`